COMP2200/COMP6200 Practical Exercise (Week 9): Tic–Tac–Toe Without the Landmines

School of Computing

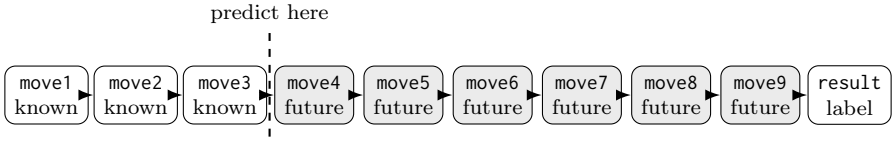
Preparation

Choose your environment: Google Colab or Jupyter on your own computer.

Part A — Hands-on: what are we predicting? (10 min)

Data: start with `tictactoe.csv`. It is a small sample with the same columns as the full file, so it is easier to inspect by hand.

1. Pick one row and play that game on paper. Move 1 is X, move 2 is O, and so on.
2. Stop after `move3`. Based only on the first three moves, guess whether the final result will be `X win`, `O win`, or `draw`.
3. Now reveal the rest of the row and check the actual result.
4. Count the result column. Which class would a boring "always guess the most common result" model choose?

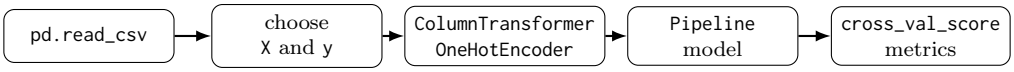


The question for today: can we build a supervised model that predicts the result from the opening, and can we notice when we accidentally use information from after the prediction time?

Part B — Python: build the supervised pipeline (45 min)

Data: now use `all_tictactoe_games.csv` (columns `move1`–`move9`, target `result`).

The snippet below turns the question from Part A into the same supervised pipeline shape we used in the lecture: load the data, separate `X` and `y`, preprocess categorical columns, train a model, and evaluate it with cross-validation.



Usually the first cell has the imports. We'll find ourselves using most of these. (Normally you would add to the imports list as you need them.)

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

First we need to load the data. Set the filename to match where you saved it.

```
data = pd.read_csv('where/you/downloaded/all_tictactoe_games.csv')
```

Start with the target variable. What are we trying to predict?

```
y = data['result']
```

We can now double-check the distribution of wins and losses.

```
y.value_counts()
```

What are we trying to predict `y` with? Start with the honest version: only the first three moves.

Initially, let's see if we can predict the conclusion of a game using the first three moves. Later we'll see if we can do it with more or less.

```
X = data[['move1', 'move2', 'move3']]
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=0)
```

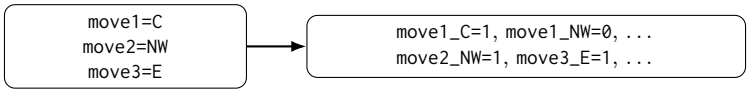
Let's establish a baseline. How well could you perform if you just guessed?

Create a dummy classifier, and then get the cross-validation score for accuracy and the F1 score.

```
dummy = DummyClassifier()
print('Dummy accuracy:' , cross_val_score(dummy, X, y, cv=cv).mean())
print('Dummy macro F1:' , cross_val_score(dummy, X, y, cv=cv, scoring='f1_macro').mean())
```

The `X` data is categorical. We need to turn it into numbers if we want to do something smarter than a dummy classifier.

How does one-hot-encoding work? How many columns will we end up with? Is that a reasonable amount?




```
def make_pipeline(columns):
    preprocess = ColumnTransformer([
        ('ohe', OneHotEncoder(handle_unknown='ignore'), columns)
    ])
    return Pipeline([
        ('prep', preprocess),
        ('clf', LogisticRegression(max_iter=500))
    ])
```

There are no missing data points, so we don't need to impute any missing values. We don't need to scale the data either here. (Why not?)

So that `ColumnTransformer` is all that is needed for the tic–tac–toe data. We wrap it in a small function because later we want to compare a three-move model with a nine-move model.

Now build the opening-only pipeline.

```
pipe = make_pipeline(X.columns)
```

Calculate and print out a cross-validated accuracy and score for the pipeline you have just created.

```
acc = cross_val_score(pipe, X, y, cv=cv, scoring='accuracy')
f1 = cross_val_score(pipe, X, y, cv=cv, scoring='f1_macro')
print('Accuracy:', acc.mean())
print('Macro F1:', f1.mean())
pipe.fit(X, y)
```

Now repeat the same calculation with all nine move columns:

```
all_moves = [f'move{i}' for i in range(1, 10)]
X_all = data[all_moves]
pipe_all = make_pipeline(X_all.columns)
acc_all = cross_val_score(pipe_all, X_all, y, cv=cv, scoring='accuracy')
f1_all = cross_val_score(pipe_all, X_all, y, cv=cv, scoring='f1_macro')
print('All-move accuracy:', acc_all.mean())
print('All-move macro F1:', f1_all.mean())
```

If the all-move score is much higher, be suspicious. Later moves are not available if you are really predicting after move 3.

Question: what happens if you try to predict the outcome using more columns, or less columns? Why might more columns be leaking data? (i.e. why might later columns make it no longer a prediction?)

It's helpful to understand how it's making its classifications. For that, we need to know what weights it's putting on each feature, and what the final feature names after the preprocessing steps have run.

Let's find the feature names first.

```
prep_step = pipe.named_steps['prep']
feature_names = prep_step.get_feature_names_out()
print(feature_names)
```

Now let's get the coefficient weights. There is one row per class, so we loop over the class names and the coefficient rows together:

```
classifier = pipe.named_steps['clf']
for class_name, coefficients in zip(classifier.classes_, classifier.coef_):
    strengths = pd.Series(index=feature_names, data=coefficients)
    print(class_name)
    print("Strongest positive weights")
    print(strengths.nlargest(10))
    print("Strongest negative weights")
    print(strengths.nsmallest(10))
```

Permutation importance asks a slightly different question: what happens to the score if we shuffle one input column and leave the rest alone? Use a sample so it runs quickly.

```
importance_data = data.sample(n=20000, random_state=0)
importance = permutation_importance(
    pipe,
    importance_data[['move1', 'move2', 'move3']],
    importance_data['result'],
    scoring='f1_macro',
    n_repeats=5,
    random_state=0
)
move_importance = pd.Series(
    index=X.columns,
    data=importance.importances_mean
)
print(move_importance.sort_values(ascending=False))
```

Part C — From Tic–Tac–Toe to Penguins (if time permits)

A reference notebook shows how to handle `penguins.csv` with imputation and scaling. Recreate that process:

1. Copy your working Python notebook from Part B.
2. Replace the tic–tac–toe dataset with `penguins.csv`.
3. Insert preprocessing steps:
   (a) impute missing numeric values and scale them,

```
num_pipe = Pipeline([
    ('impute', SimpleImputer()),
    ('scale', StandardScaler())
])
```

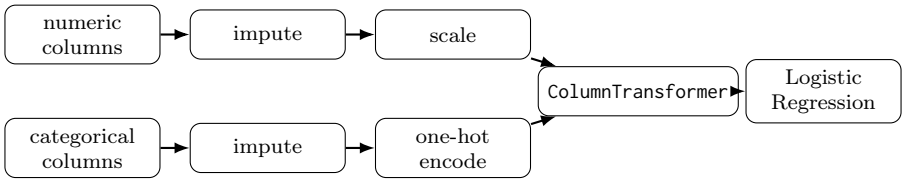
(b) impute categorical values and one-hot encode them.

```
cat_pipe = Pipeline([
    ('impute', SimpleImputer(strategy='most_frequent')),
    ('ohe', OneHotEncoder(handle_unknown='ignore'))
])
```

(c) combine the numeric and categorical preprocessing.

```
preprocess = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols)
])
```

4. The same per-class coefficient idea applies here too: penguins has three species, so look at each class name alongside the matching row of `coef_` instead of assuming `coef_[0]` is the whole model.



Part D — Model comparison and tuning

What happens if you replace the Logistic Regression object with another classifier that we have learned? Can you make a `GridSearchCV` to try a variety of hyperparameters?

Part E — Practice quiz

There are practice quiz questions as usual. Check iLearn for the current assessment announcements and due dates.